

NSASAG 2026

Contents

NSASAG 26-01	A Batched Variant of a Coupon-Collector Problem	2
NSASAG 26-02	How to Approach a Binary Classification Problem	4
NSASAG 26-03	Verbatim Memorization of Text by LLMs	6
NSASAG 26-04	Discovery of the Largest N -Dimensional Parallelotope Inscribed in a Cluster of Hypercubes	8
NSASAG 26-05	Diagnosing and Correcting HDBSCAN Mergers for Latent-Mean Recovery Under a Gaussian-Mixture-Plus-Clutter Model	9
NSASAG 26-06	Recognizing Local Hot Spots in Tails of Distributions	11
NSASAG 26-07	Bayesian Location Estimation	13
NSASAG 26-08	A Visualization Problem	16
NSASAG 26-09	Dynamic Graph Clustering Revisited	17
NSASAG 26-10	Intermittent Sparse Signals Associated with Events	19
NSASAG 26-11	Questions on the Inner Product Laplacian Revisited	21
NSASAG 26-12	Multi-layer Network Clustering	24

NSASAG 26-01 A Batched Variant of a Coupon-Collector Problem

Consider a scenario where each cereal box has a probability p of containing a coupon, independently. Our goal is to accumulate $s \geq 1$ coupons in order to win the Grand Prize, where s is given parameter. The parameter $p \in [0, 1]$ is completely unknown.

If we buy cereal boxes one-by-one, this requires s/p time on average. Note that this strategy does not depend on knowing p (although the cost does). The downside is that this strategy is completely sequential. If we (approximately) knew p , then we could simply buy $\approx s/p$ cereal boxes in one go. A standard argument via the multiplicative Chernoff bound shows that this gives us the desired s coupons with good probability.

Our question is whether it is possible to find a strategy where we buy the boxes in large batches (each of which can be done in parallel, say), while simultaneously minimizing the total number of boxes, all without any knowledge or assumptions about p .

Concretely, we would consider a scheme as follows. In each iteration i , we choose a batch size B_i and buy B_i cereal boxes. We then learn the resulting number of coupons M_i , where $M_i \sim \text{Binomial}(B_i, p)$. The batch sizes B_i can be chosen adaptively based on all previous values $M_1, \dots, M_{i-1}$. We stop the process at the first time T when $M_1 + \dots + M_T \geq s$.

We would like to minimize the expected stopping time $\mathbb{E}[T]$, while also ensuring that $\mathbb{E}[\sum_{i=1}^T B_i]$ is close as possible to the sequential optimum s/p . If we want to be concrete, we can consider minimizing an objective function of the form $\mathbb{E}[\sum_{i=1}^T (\lambda + B_i)]$ for a given parameter λ . Here, λ represents the fixed cost for starting a batch, which we pay independently of the number of boxes in the batch.

We would be primarily interested in a frequentist approach, so to minimize regret irrespective of p . Here, the regret would be the cost of our strategy, divided by the cost of the optimal sequential strategy s/p . We don't want to assume any Bayesian priors on p .

We are aware of a few possible strategies for this problem. The first is to choose the batch sizes B_i geometrically. Concretely, we set $B_i = \alpha^i$, for some constant $\alpha > 1$. There are two problems with this scheme: (1) The number of batches may be relatively large, $T = \log_\alpha(s/p)$; (2) due to overshoot effects, the total number of cereal boxes may be $(2 + \alpha)$ times the sequential optimum.

Another scheme is to increase B_i by a factor of $c_1 s$ in each iteration, until the total number of coupons exceeds some threshold c_2 . (Here c_1, c_2 are constants). If we have $N = B_1 + \dots + B_k$ boxes up to this point and achieved $S = M_1 + \dots + M_k \geq c_2$ coupons, we then estimate crudely $\tilde{p} = S/N$. For all subsequent batch sizes we set $B_i = (s - S)/\tilde{p}$. This scheme should take $O(1 + \log(1/p)/\log(s))$ batches (much better), and the number of boxes should be much closer to the optimum for c_1, c_2 appropriately chosen.

For the second scheme, we are not sure how to choose c_1, c_2 appropriately. Also, this two-phase scheme seems suboptimal, we are wondering if there is a cleaner strategy.

As an advanced variant of this problem, we can consider a situation where the batch cost λ itself depends on the unknown p , i.e. a cost of the form $\sum_{i=1}^T (\lambda(p) + B_i)$.

We are also interested in a variant where we don't specify an *exact* batch size B_i , but instead specify a rate Z_i , and set $B_i \sim \text{Poisson}(Z_i)$. We learn B_i as well as M_i (which is determined from B_i). It stands to reason that this should have a negligible effect on the process, but we would like a rigorous analysis.

NSASAG 26-02 How to Approach a Binary Classification Problem

We need to develop an interpretable model to perform binary classification of objects. The training data consist of about 500 objects which belong to the positive class (certainly) and about 2000 objects which belong to the negative class (mostly — a few may actually be of the positive class). The test data consist of about 20000 objects to classify.

The i^{th} object is described by a fixed-length *feature vector* $v^i = (v_1^i, \dots, v_b^i)$ and a variable-size *record set* $R^i = \{r_1^i, \dots, r_{n_i}^i\}$, where each *record* is itself a fixed-length vector $r_j^i = (r_{j,1}^i, \dots, r_{j,c}^i)$.

The length of the feature vectors is $b \approx 100$. The coordinates of the feature vectors v^i can be continuous or categorical and each categorical coordinate can take 2 to 10 values.

The size of the record set R^i is $n_i \approx 10^6$ or 10^7 and the length of the each record is $c \approx 1000$. The coordinates of the record vectors r^i can be continuous or categorical and each categorical coordinate can take 2 to 100 values.¹ The record set R^i reflects observations about certain properties of the i^{th} object.

- Each object is a collection of hundreds or thousands of *components*.
- There are tens of *observation methods* that can be applied to a component.
- Upon application, an observation method assigns values to a method-specific subset of the c record coordinates.
- Not all observation methods apply to all components.

A *record* r_j^i is the result of applying an observation method to a component of the i^{th} object. If we were to arrange the record set into an $n_i \times c$ matrix, some of the columns would be complete while others would be quite sparse (that is, mostly missing) or even empty. The sparsity is such that the full training data set only takes up 1-2TB (compared to between 9 and 90TB if all features were present and single-precision floating-point numbers). We do not know, outside of the data, what components a given object may have or what component and observation method produced a given record — but we hope that strong indications of these are present in records themselves.

We seek practical advice on how to approach this problem: that is, recommended methods for performing EDA and feature engineering to produce an accurate, interpretable binary classifier. The proposers of this problem all have graduate-level training in statistics and are curious to know, among other things, how students at the NSASAG schools are being taught to handle problems like this one. For example, is human feature-engineering still done, or has it been partially or completely automated via machine learning? While we need an interpretable model as the final product, we are open to using less-interpretable methods to discover useful structure in the data as an intermediate step.

¹For a slightly more difficult version of this problem, consider the case where some of the categorical variables are recorded with coding errors such that a given category may appear in the data under several distinct names.

Domain knowledge regarding the feature vectors, components, and observation methods is theoretically available but extremely expensive. The effort required to develop thorough understanding of all of the types of object components and all of the observation methods would be many times the value of a single trained classifier. Domain knowledge needed to understand the meaning of an interpretable model will, of course, be obtained (and model-guided acquisition of domain knowledge can be used iteratively to improve a model).

To make the ideas in the abstract statement of the problem more concrete, it may help to consider an example where objects are zoos, where: features capture information like area, location, staffing, and budget; components of objects are individual animals; and observation methods include anything that zoos use (for example, feeding or activity logs of zookeepers, medical records of veterinarians, perhaps extracts from video footage). The binary label present in the training data might indicate whether the zoo follows a specific set of practices.² A record field that records the date of the observation would usually or always have a value, but a record field that records tree-climbing activity would only contain a value in records corresponding to animals that are capable of climbing trees (and perhaps only if the observed animal is actively climbing during the observation).

²For example, adherence to the World Association of Zoos and Aquariums' *Code of Ethics and Animal Welfare*.

NSASAG 26-03 Verbatim Memorization of Text by LLMs

As LLM input context size and model capacity grows, we are interested in a comprehensive (statistical or geometric) description of the way LLMs store and return verbatim content upon prompting through an LLM agent. The recent work, Ahmed et al. “Extracting Books from Production Language Models” [arXiv:2601.02671], demonstrates the powerful memorization mechanism in practice. Also suggestive of compression aiding the memorization mechanism is semantic superposition [arXiv:2505.10465, arXiv:2209.10652] as well as a theoretical analysis of the high ‘succinctness’ of transformers [arXiv:2510.19315].

The Ahmed et al. paper in particular demonstrates an efficient method of verbatim elicitation from the LLM/LLM agent of a particular type of memorized data, i.e. literary works, and several methods of measuring the quality of the returned verbatim data. This work in particular inspires numerous questions and refinements. With regard to the memorization mechanism and compression, we pose several questions:

- What are the implied (natural language) methods to systematically elicit other types of verbatim data from both open-weights and frontier LLMs or LLM agents?
- What are the legal and ethical implications with regard to copyrighted or otherwise protected (e.g., HIPAA) text stored in an LLM?
- What roles do guardrails in a chat agent play to enhance or suppress evidence of memorization and the efficiency of verbatim elicitation from the model?
- Can a general proof (under some stated assumptions) give a positive or negative answer to whether this memorization is an inevitable general feature of transformer-based LLMs through available chat APIs? If negative, how do specific architecture qualities affect these mechanisms?
- We are also curious if and how the above assertions and questions also apply to available large enough MAMBA-based (recurrent) LLMs [arXiv:2312.00752].

To clarify definitions for this problem:

- C is a context consisting of a concatenation of tokens $\{t_0, t_1, \dots, t_N\}$, one or more characters encoded for machine consumption from a specified finite set of possible tokens.
- An *LLM* is a deterministic trained function from context to logits capturing the output p.d.f. of the next token, $\text{LLM} : C \rightarrow \mathbb{R}^n$.
- An *LLM agent* is any program (e.g. API) that calls an LLM as a subroutine, e.g. a chat agent iteratively calls an LLM given context then uses the output logits to sample the next token until message termination.

References

1. A. Ahmed, A. F. Cooper, S. Koyejo, P. Liang, “Extracting books from production large language models,” arXiv: 2601.02671 (2026).

2. Y. Liu, Z. Liu, J. Gore, “Superposition Yields Robust Neural Scaling,” arXiv: 2505.10465 (2025).
3. N. Elhage, et al., “Toy Models of Superposition,” *Transformer Circuits Thread*, arXiv: 2209.10652 (2022).
4. P. Bergsträßer, R. Cottrell, A. W. Lin, “Transformers Are Inherently Succinct,” arXiv: 2510.19315 (2025).
5. A. Gu, T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” arXiv: 2312.00752 (2023).

NSASAG 26-04 **Discovery of the Largest N -Dimensional Parallelotope Inscribed in a Cluster of Hypercubes**

Consider an N -dimensional Euclidean space which is the disjoint union of N -dimensional unit hypercubes. Two hypercubes are adjacent if they share an $N - 1$ dimensional facet. Each hypercube has a label, *negative* or *positive*.

A *cluster* is a maximal set of adjacent positive hypercubes. That is, the interior of the union of the hypercubes defining the cluster is connected. No particular shape or structure to the cluster is assumed. Define a *box* to be an N -dimensional parallelotope that has all of its facets perpendicular to the axes of the N -dimensional space and which is equal to a union of positive labeled hypercubes.

The challenge is to find the largest box that fits within a given cluster. Estimating the volume of such a box is not helpful; what is needed is the box itself. If finding the largest box is intractable, we are interested in methods for finding one or more large boxes, even if there is no guarantee of maximum volume.

We are most interested in approaches that will scale to spaces of high dimension ($N > 500$) and clusters containing a large number of hypercubes (> 1 million), but progress on smaller problems is of value as well.

References

Some potentially related papers include:

1. *Computing the Volume is Difficult*, Imre Bárány and Zoltán Füredi, ACM, 1986.
2. *Inner and outer approximations of polytopes using boxes*, Alberto Bemporad, Carlo Filippi, and Fabio D. Torrisi, Computational Geometry Theory and Applications, 2004.
3. *On the Complexity of Four Polyhedral Set Containment Problems*, Robert Freund and James Orlin, Mathematical Programming, 1985.

NSASAG 26-05 Diagnosing and Correcting HDBSCAN Mergers for Latent-Mean Recovery Under a Gaussian-Mixture-Plus-Clutter Model

Setting

Let $D \subset \mathbb{R}^2$ be a bounded observation region. We observe a point configuration

$$\mathcal{X} = \{X_{0j}\}_{j=1}^{M_0} \cup \bigcup_{i=1}^N \{X_{ij}\}_{j=1}^{M_i},$$

where:

- $X_{0j} \stackrel{\text{iid}}{\sim} \text{Unif}(D)$, ($j = 1, \dots, M_0$), with M_0 possibly Poisson,
- for $i = 1, \dots, N$, $X_{ij} \mid M_i \stackrel{\text{iid}}{\sim} \mathcal{N}(A_i, \Sigma_i)$, $j = 1, \dots, M_i$,
- $A_i \in \mathbb{R}^d$ are unknown component means,
- Σ_i are unknown arbitrary covariance matrices,
- M_i are random component sizes, for example $M_i \sim \text{Poisson}(\lambda_i)$, with unknown $\lambda_i > 0$,
- the number of components N is unknown.

Alternatively, one may regard the observed data as a realization of a spatial point process with intensity

$$f(x) \propto \eta_0 \mathbf{1}_D(x) + \sum_{i=1}^N \eta_i \phi_d(x; A_i, \Sigma_i),$$

where $\phi_d(\cdot; A_i, \Sigma_i)$ denotes the d -variate Gaussian density. Our inferential target is the collection of latent means $\{A_1, \dots, A_N\}$.

Motivating practice

A common empirical procedure is:

1. Apply HDBSCAN with default settings to the observed point cloud $\mathcal{X}$,
2. Treat each returned cluster as representing one latent component,
3. Estimate the corresponding A_i by the empirical centroid of that cluster.

However, this strategy can perform poorly when multiple Gaussian components are sufficiently close, anisotropic, or linked by moderate-density regions, so that HDBSCAN returns a single connected cluster containing points from several latent components. In such cases, the empirical centroid of the merged cluster may be far from every true mean A_i , and may even lie in a low-density region with few nearby observations.

Problems

1. Population-level Merge Conditions. Under the model above, characterize conditions on $\{A_i, \Sigma_i, \lambda_i\}_{i=1}^N$ and the background level under which HDBSCAN's population target would merge multiple Gaussian components into a single cluster. Since HDBSCAN is a density-based procedure, one natural formulation is in terms of connectivity of density upper level sets: $L_t = \{x \in D : f(x) \geq t\}$. We seek conditions under which distinct Gaussian components correspond to distinct connected components of L_t , and conditions under which they are connected by a density bridge for the range of levels relevant to HDBSCAN.
2. Practical diagnostics for merged components Can we build diagnostics, computable from a single observed dataset, that flag HDBSCAN clusters likely to contain points from multiple latent Gaussian components. Of particular interest are diagnostics that detect:
 - Non-elliptical or highly curved cluster geometry,
 - Internal multimodality,
 - Graph bottlenecks or weak bridges in the mutual-reachability graph,
 - Cases in which the cluster centroid is geometrically implausible as an estimator of any latent mean.
3. Improved estimation of A_i after clustering When HDBSCAN merges several latent components into one cluster, determine whether one can improve upon the cluster centroid as an estimator of the underlying means A_i . We've built methods based on post-processing with another clustering algorithm, but would appreciate guidance on something that is less ad-hoc.

NSASAG 26-06 Recognizing Local Hot Spots in Tails of Distributions

We study a rare-subpopulation enrichment problem in $\mathbb{R}^3$. Let

$$X_1, \dots, X_n \stackrel{\text{iid}}{\sim} f_X, \quad Y_1, \dots, Y_m \stackrel{\text{iid}}{\sim} f_Y,$$

where X denotes a background population and Y a focal subpopulation, with $n \gg m$ (for example, $n \approx 10^6$ and $m \approx 10^2$). Our goal is to identify localized hot spots where the focal subpopulation is over-represented relative to background, as quantified by the density ratio

$$r(z) = \frac{f_Y(z)}{f_X(z)}, \quad z \in \mathbb{R}^3.$$

In our applications, a hot spot is defined to be a local maximum of $r(z)$ whose value exceeds a threshold τ , which needs to be determined empirically from the data. In some situations, we can add a minimum-density constraint to exclude regions where the background density $f_X(z)$ is too small for the ratio $r(z)$ to be stable.

Operationally, we evaluate the relevant estimators on a grid and report either a ranked list of hot spot locations, a thresholded enrichment map, or a small set of top peaks, depending on the use case. Our current approach is to estimate f_X and f_Y separately using kernel density estimators, intentionally applying heavier smoothing to the background density f_X and lighter smoothing to the focal density f_Y , and then forming a plug-in estimate of $r(z)$. This asymmetric smoothing has been empirically useful: we want a stable, slowly varying denominator together with sensitivity to sharp local structure in the numerator. However, both smoothing and threshold selection have so far been largely ad hoc. We are therefore interested in principled statistical guidance on the following questions:

1. *Hot spot targeted estimation.* How should one estimate $r(z)$ when the inferential target is hot spot detection rather than globally accurate estimation of the ratio surface? In particular, can one formulate loss functions that prioritize correct identification, localization, and ranking of local maxima of $r(z)$ above threshold?
2. *Bandwidth or regularization selection.* If one continues with a KDE-based plug-in approach, how should smoothing or regularization parameters be selected when $n \gg m$ and the scientific goal is detection of sharp peaks in r rather than optimal estimation of f_X and f_Y separately? More specifically, is there a principled way to choose a deliberately smooth baseline estimate of f_X together with a less-smoothed estimate of f_Y ?
3. *Reducing tuning complexity.* Our current workflow involves at least two smoothing parameters and a detection threshold. Are there principled ways to reduce or couple these tuning parameters — for example, deriving threshold choices from the smoothing scale or replacing multiple tuning parameters by a single regularization framework?
4. *Local inference and multiple testing.* How can one test local hypotheses of the form

$$H_0(z) : r(z) \leq 1 \quad \text{versus} \quad H_1(z) : r(z) > 1,$$

while identifying hot spot peaks and controlling the false discovery rate across candidate grid locations or detected local maxima?

5. *Robustness to extreme dynamic range.* How should estimation and inference be stabilized when the background density $f_X(z)$ varies over several orders of magnitude, especially in regions where $f_X(z)$ is very small and plug-in ratio estimates may become unstable or spuriously large?
6. *Direct density-ratio estimation.* Under what regimes in n , m , signal sharpness, and background heterogeneity is direct estimation of $r(z)$ preferable to separately estimating $f_X(z)$ and $f_Y(z)$ and then taking their ratio?
7. *Alternative frameworks.* Beyond KDE-based density-ratio estimation, are there other methods recommended for this setting — for example, direct density-ratio estimation, classifier-based methods, local two-sample testing, nearest-neighbor methods, scan statistics, or point-process/intensity-ratio models?

Although we have written the problem in i.i.d. density form, some of our motivating applications are naturally spatial or spatiotemporal, so we would also be interested in whether a point-process formulation in terms of intensity ratios would be more appropriate.

NSASAG 26-07 Bayesian Location Estimation

Problem Setup

We estimate the location of a stationary item using distance measurements collected from multiple scan locations. The problem is formulated as follows:

- $\theta \in \mathbb{R}^2$: Unknown location (longitude, latitude)
- $x_i \in \mathbb{R}^2, i = 1, \dots, n$: Known scan locations where measurements are taken
- $d(\cdot, \cdot) : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}_+$: Distance function (e.g., Euclidean or geodesic)
- $d_i \in \mathbb{R}_+, i = 1, \dots, n$: Observed distance measurements from each scan location to item

The true distance from θ to scan location x_i is $d(\theta, x_i)$. However, observed distances d_i contain measurement error due to positioning uncertainty and instrument noise. Our goal is to infer the posterior distribution $p(\theta \mid \mathbf{d})$ where $\mathbf{d} = (d_1, \dots, d_n)$.

Hierarchical Error Model

We employ a two-layer hierarchical model to capture positioning error and measurement error separately:

$$\theta \sim \text{Prior}(\theta) \tag{1}$$

$$\mu_i \mid \theta, x_i \sim \text{TruncNormal}(d(\theta, x_i), \tau^2, 0, \infty) \tag{2}$$

$$d_i \mid \mu_i \sim \text{Gamma}(\text{mean} = \mu_i, \text{variance} = \sigma^2) \tag{3}$$

The latent variable μ_i represents the “true” distance after accounting for positioning error but before measurement noise. The truncation at zero ensures $\mu_i > 0$ as required by the Gamma distribution.

Model Components

Layer 1 (Positioning error): The truncated normal prior (2) models uncertainty in the position relative to scan location x_i . The hyperparameter τ^2 controls this positioning uncertainty. The truncation normalization constant is $Z(\theta, x_i, \tau^2) = \Phi(d(\theta, x_i)/\sqrt{\tau^2})$ where $\Phi(\cdot)$ is the standard normal CDF.

Layer 2 (Measurement error): The Gamma distribution (3) models instrumental measurement noise with fixed variance σ^2 . We parametrize the Gamma distribution by its mean μ_i and variance σ^2 , giving shape $\alpha_i = \mu_i^2/\sigma^2$ and rate $\beta_i = \mu_i/\sigma^2$.

Prior

Prior on θ

We use a uniform prior over a rectangular search region:

$$p(\theta) = \begin{cases} \frac{1}{A} & \text{if } \theta \in [a, b] \\ 0 & \text{otherwise} \end{cases}$$

where $A = \prod_{j=1}^2 (b_j - a_j)$ is the area of the region $[a, b] = [a_1, b_1] \times [a_2, b_2]$.

Hyperprior on τ^2

While τ^2 is nominally a hyperparameter, we estimate it empirically from the data using marginal likelihood maximization:

$$\hat{\tau}^2 = \arg \max_{\tau^2} p(\mathbf{d} \mid \theta_{\text{PSO}}, \tau^2, \sigma^2)$$

where θ_{PSO} is a point estimate from particle swarm optimization with objective function

$$\min_s \sum_{i=1}^n (d(x_i, s) - d_i)^2,$$

and the marginal likelihood is computed by integrating out the latent variables $\{\mu_i\}$ using Gauss-Hermite quadrature.

We test four candidate estimators derived from data statistics (empirical variance of residuals, safety-scaled $(\min d_i/3)^2$, range-based, and marginal likelihood) and select the one that maximizes marginal likelihood while ensuring truncation safety: $d_{\min}/\sqrt{\hat{\tau}^2} \geq 3$ where $d_{\min} = \min_i d(\theta_{\text{PSO}}, x_i)$.

The fixed measurement variance σ^2 we currently choose a priori based on best guess.

Posterior

The marginal posterior after integrating out $\{\mu_i\}$ is:

$$p(\theta \mid \mathbf{d}) \propto p(\theta) \prod_{i=1}^n \int_0^\infty p(d_i \mid \mu_i) p(\mu_i \mid \theta, x_i) d\mu_i$$

We sample from the posterior using Metropolis-Hastings MCMC with a truncated normal proposal distribution:

$$q(\theta^* \mid \theta) = \text{TruncNormal}(\theta^*, \theta, \delta^2 I, a, b)$$

where δ is the proposal step size (tuned to achieve 20–50% acceptance rate), I is the identity matrix, and $[a, b]$ are the prior bounds. We run multiple chains with starting values θ_{PSO} plus random error.

Questions

We are interested in determining what improvements can be made; we would like to have our credible region as small as possible given our restraints (such as σ^2 may not be a value we can control) while still properly representing $100 \times (1 - \alpha)\%$ credibility.

1. What would be some best practices to choose the start points of each MCMC.
2. Since we have some rough idea of θ given $d_1, \dots, d_n$ is there a data driven way to leverage this into our prior?
3. Distribution of μ_i ; are there other symmetric distributions that maintain the truncation and/or result in a posterior that does not require numerical integration?
4. Are there other data driven approaches to estimate τ^2 we could use? Would addition of a hyperprior be advantageous?
5. What if the value of τ^2 is known conditional on location?
6. What if σ^2 is not constant, but known, for all locations?
7. What if σ^2 is unknown but we know some functional relationship of the σ^2 based on location?
8. If σ^2 are not constant and not known what are ways we can come up with data driven approximations or hyperprior(s)?
9. Generating from the posterior using something other than Metropolis-Hastings?

NSASAG 26-08 A Visualization Problem

We have counts of 10-20 different things over a two-dimensional region, both before and after an event of interest. We would like to locate and describe large changes.

Due to factors outside of our control, we need to visualize everything in one to three maps which need to be understandable by consumers who are experts in an applied domain but not experts in statistics. In particular, more dramatic changes should show up on the map(s) more clearly than less dramatic changes, but we are not asking for a hypothesis test of whether a change is “real” or “noise”.

As a concrete example, consider the case where we have data on how many plants of each of 10 plant species are present in each acre of a state park (from, say, satellite imagery), and tracks of deer, elk, and wolves based on radio collars. Comparing data from 10 years ago to data from this year, we’re interested in visualizing how the environment has changed.

While we have a strong preference for visualizations that can be implemented with standard Python or R packages, we have the resources to implement a custom visualization if there is a compelling reason to do so. We are open both to new research and to a literature and software review with recommendations.

NSASAG 26-09 Dynamic Graph Clustering Revisited

Dynamic graph clustering has been a recurring theme in NSASAG problems since 2008 and static graph clustering even earlier. This current problem is an updated variant of NSASAG 2018-14, which is related to earlier questions including NSASAG 17-04, NSASAG 15-12 *Evaluation of Dynamic Community Detection Algorithms*, and NSASAG 13-14 *Evaluation of Dynamic Community Detection Algorithms*.

Background.

A *dynamic graph* $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathcal{T})$ is a collection of edges $\mathcal{E}$ between vertices $\mathcal{V}$ with temporal metadata $\mathcal{T}$ living on the edges and/or vertices. Here, consider the simplest version where each edge is of the form (i, j, t) , $i, j \in \mathcal{V}$, $(i, j) \in \mathcal{E}$, and $t \in \mathcal{T}$ is a real-valued *timestamp* in $[0, T]$ for (i, j) . Also, assume that the graph is simple and undirected.

Communities in static graphs are vertex subsets $\mathcal{S} \subset \mathcal{V}$ that notionally are densely connected within $\mathcal{S}$ and more sparsely connected within $\mathcal{V} \setminus \mathcal{S}$. Consider finding such communities within a dynamic graph and finding interesting dynamic structural changes of these communities, such as (i) a community being born/dying, (ii) a community growing/reducing, (iii) a community changing its internal structure, and (iv) two or more communities splitting/merging.

One of the ongoing questions for analyzing dynamic graphs is the determination of time steps to partition a graph into a sequence of temporal slices. This was a question in NSASAG 18-14 and again this year. Often community detection algorithms require the choice of a timestep width used to partition a dynamic graph into a sequence of discrete graphs each representing a Δt -wide segment of the total time T . If Δt is too large, important dynamics are lost, whereas, if Δt is too small, techniques often cannot see any community structure at all because the time-aggregated graphs are highly disconnected. Additionally, streaming algorithms may require a “burn-in” period Δt_0 where additional observations are needed at the start of the dynamic graph before community structure can be detected. A streaming algorithm will need to take into account factors such as community births/deaths, community growth/reduction, internal structural changes, and communities splitting/merging.

We make several comments on some related recent dynamic graph research. Badlani et al (2018) incorporate temporal information into standard static graph algorithms such as the Louvain and Leiden algorithms, preferring the Leiden algorithm based on their experimentation. They create subgraphs choosing generic time slices of approximately two weeks, one month, and three months. Then they use a time weighted conductance across time slices to measure how community structure changes over time. As another example of time slice determination, Zhong et al (2025) use windowed modularity to determine time slices and propose two extensions of existing mutual information measures to evaluate community structures for temporal networks. Finally, Christopoulos and Tsihlias (2022) present a high level survey of current research into time-evolving networks.

Badlani et al (2018) define two weight functions (linear and exponential) and use them to determine a time weighted adjacency matrix for each time step. The weighted adjacency matrix for a fixed time step is defined as follows. Let $w_{ij}(t)$ represent the selected weight function for an edge between nodes i and j that occurs at time t . The weight $w_{ij}(t) = 0$ if there is no edge between i and j at time t . Then,

$$a_{ij} = \sum_t w_{ij}(t),$$

which reduces to the standard adjacency matrix without a weight function.

Problem Statements.

- I. Estimate both the burn-in period Δt_0 , and the time interval Δt used in order to break a graph into time slices of width Δt . Is it possible to accommodate different rates of change in the network, i.e. some regions change within a short time-scale while other regions have a long time-scale?
- II. Does community detection actually require the determination of a time step Δt or is it possible to perform community detection as the edges occur without specifically determining Δt ?
- III. Is there a better way to incorporate time into the algorithm instead of using the time weighted adjacency matrix? Is there an alternative weight function that should be considered?
- IV. Because it takes into account the degree distribution of the nodes, the proposer of this problem prefers the use of the normalized Laplacian instead of the conductance to determine the quality of a community. Is the time weighted normalized Laplacian a better statistical alternative to the time weighted conductance?

References.

- R. Badlani, K. Culberg, and Z. Jiang, "Community Detection and Evolution in Temporal Networks," Stanford Network Analysis Project (SNAP), 2018.
- K. Christopoulos, and K. Tsichlas, "State-of-the-art in Community Detection in Temporal Networks," `AIAI_2022_Survey_Temporal_Comm_Detection`, 2022.
- P. Zhong, C. Ba, R. Mondragon, and R. Clegg, "Quantifying community evolution in temporal networks," *Scientific Reports* 15, 2025.

NSASAG 26-10 Intermittent Sparse Signals Associated with Events

Motivating Example.

Note: This problem is a generalization of NSASAG 23-05, “Correlation of Extremely Sparse Candidate Signals.” Consider a city and the road construction events in that city over the course of several years. Each construction event is largely independent. For example, different construction companies might be active over this time span, or different organizers and crews within a single company. Different conditions might prevail, like highway work vs. city-central work, winter vs. summer, etc. However, we consider the hypothesis that there is a general, observable sequence of micro-events that lead up to the macro-event of a road construction start. Some variant of warning devices like traffic cones or flags or signage might appear, then capital equipment might arrive, then work crews might start setup, then finally the work begins. Suppose that during this process a stream of public observations arises, like posts to social media noting changes in condition at certain intersections or mile markers. These observations arise from sources that might or might not persist from one construction event to the next, like tourists or transient drivers.

Consider each suitably-preprocessed social media account to be the source of a possible signal (e.g., if some word in the lexicon of road construction is used, generate an observation). This signal will typically be sparse. There are many signals. Is it possible to use these sparse signals to predict the events? NSASAG 23-05 supposed that signals persist across the whole time horizon. Now we recognize that signals may come and go. We seek solutions resilient to this transient behavior. Our problem becomes one of identifying meta-parameters that bind a family of models (one per event). We have a history of k known events in the time span. Can we use that history to more effectively predict future event $k + 1$?

Formalization.

We have K events, each described by parameters $(\tau, \eta, \theta)_k$, $k \in \{1, 2, \dots, K\}$. Each event is associated with a dataset of count data $X^{(k)}_{n \times p}$, where each row is a measurement in time and each column contains the observations of a single signal. Following the n^{th} measurement of $X^{(k)}$, event k occurs. Each dataset has η_k unknown “active” columns/variables I_k whose indices are sampled uniformly such that

$$|I_k| = \eta_k \ll p$$

and

$$I_k \subset \{1, 2, \dots, p\}.$$

This index sampling is independent across the K events, so a variable might or might not be active for more than one event. Typically, n is approximately 1000 epochs, p about 100K signals, and η_k dozens (the signals that react to a localized construction event (a small fraction of the city)).

The active columns of datasets $X^{(k)}$ exhibit a mean shift such that $X_{ij}^{(k)} \sim \text{Pois}(\lambda_j)$ if $j \notin I_k$, and if $j \in I_k$

$$X_{ij}^{(k)} \sim \begin{cases} \text{Pois}(\lambda_j) & \text{if } i < n - \tau_k \\ \text{Pois}(\lambda_j + \theta_k) & \text{if } i \geq n - \tau_k \end{cases}$$

where the Poisson distribution could be replaced with a different count distribution.

Thus, the parameter τ_k represents a transition point between “business as usual,” and “something has started to happen.” The parameter θ_k governs the strength of signal change when something starts to happen before event k .

NSASAG 23-05 requested methods for choosing a single I_k that best aligns with a “reference signal” that might spike at the k events (which were left implicit). Here, we dispense with the reference signal, make the events explicit, and hypothesize the existence of transition points τ_k that delimit normal traffic flow vs. precursor activity to a construction event.

Problem Statements.

1. We initially assume that $(\tau, \eta, \theta)_k$ are constant across k . Given K events, how can we best predict when event $K + 1$ will occur?
2. For a given η , θ , and K , how can we quantify the estimation error associated with a proposed estimator for the event time?
3. For a given τ , η , and θ , how large must K be to achieve a certain level of precision in the prediction of the event time?
4. What prior distribution or constraints on λ_j lead to a solvable problem?
5. Now assume that $(\tau, \eta, \theta)_k$ each follow some prior distribution. How would this affect the answers to the previous questions?
6. How would you approach this problem if we do not know that the data follows a simple mean shift, but some other unknown change in discrete distribution? For example, the distribution could be multiplicative, i.e., $\text{Pois}(\lambda_j \theta_k)$ instead of $\text{Pois}(\lambda_j + \theta_k)$.
7. How would you approach this problem if the data does not follow a Poisson distribution, but a heavily zero-inflated distribution (or hurdle model etc.)?
8. Suppose we have solved some variant of the problem above and we know $(\tau, \eta, \theta)_k$, and particularly I_k for all k . Could we use this information as input to a regression problem to predict future τ ? For example, could logistic or multinomial regression give us an expression $f(X^{(k)}\beta)$, interpreted as a vector of probabilities for τ ?

NSASAG 26-11 Questions on the Inner Product Laplacian Revisited

Background.

Note: This problem is a restatement/refinement of NSASAG Problem 2025-11, “Questions on the Inner Product Laplacian”, based on improved understanding of the theoretical and practical properties of the inner product Laplacian. One of the persistent challenges in clustering real world data is the desire to fuse combinatorial relationships between data points, semantic relationships between individual data points, and the semantic relationships between the combinatorial relations (often recorded as meta-data about the data or the combinatorial relationships). For example, consider the problem of associating researchers with specific academic specialties. One approach would be to build the co-publication graph (or hypergraph) and use any one of a number of combinatorial clustering methods to identify the clustering. Alternatively, one could use one of a number of embedding methods to either embed individual papers (corresponding to edges in the graph) based on natural-language processing of the content or embed individual researchers based on the body of work, academic advisers, etc. and apply geometric clustering methods to the embedding vectors. However, all of these approaches fundamentally neglect useful information in developing the clustering — ideally one would fuse the combinatorial data and the metadata in order to develop clusters that are more reflective of the unknown ground truth. One modern approach is to use a combinatorial neural networks (e.g., graph, hypergraph, simplicial, etc.) as the backbone of a machine learning model that predicts academic specialties. However, these approaches are notoriously computationally and data intensive and typically require labeled training data. Fundamentally, the underlying issue is the lack of a measure of cluster quality which incorporates both the combinatorial data as well as the associated metadata.

Definitions and Approach.

Recently, Aksoy and Young (2025) proposed a novel cluster quality metric which combines both structural/combinatorial data as well as metadata associated with the vertices and edges. Their approach is inspired by conductance cut quality measures from spectral graph theory. Specifically, recall that given a partition of the vertices of a graph, $(X, \bar{X})$, the conductance is defined as

$$\Phi(X) = \frac{|E(X, \bar{X})|}{\min\{\text{Vol}(X), \text{Vol}(\bar{X})\}},$$

where $E(X, \bar{X})$ is the set of edges between X and $\bar{X}$ and $\text{Vol}(S)$ is the sum of the degrees of vertices in S . Their essential insight is that by utilizing two inner product spaces that capture the similarity of the metadata, one on the vertices and one on the edges, the cardinality based measures of size in the conductance can be replaced with measures based on the similarity in the two inner product spaces, yielding a cluster quality measure that fuses structural properties and metadata similarity.

Definition (Inner Product Conductance). Let $G = (V, E)$ be an undirected graph on n

vertices and m edges and let $M_V \in R^{n \times n}$ and $M_E \in R^{m \times m}$ be positive definite matrices which define an inner product on the vertices and edges, respectively. For any non-empty set $X \subseteq V$, the inner product conductance of X is defined as

$$\Phi^{IP}(X) = \frac{\mathbf{1}_{E(X, \bar{X})}^T M_E \mathbf{1}_{E(X, \bar{X})}}{\min \{ \mathbf{1}_X^T M_V \mathbf{1}_X, \mathbf{1}_{\bar{X}}^T M_V \mathbf{1}_{\bar{X}} \}},$$

Unsurprisingly, Veldt and Young have recently shown that determining the vertex set of minimum (or maximum) inner product conductance is NP-hard. However, in Aksoy and Young (2025), it is shown that there exists a Laplacian-style matrix, which the authors call the inner product Laplacian, that satisfies a Cheeger-like inequality for the inner product conductance.

Definition (Inner Product Laplacian). Let $G = (V, E)$ be an undirected graph on n vertices and m edges and let $B \in \{-1, 0, 1\}^{m \times n}$ be a signed vertex-edge incidence matrix (that is, for every edge $e = \{u, v\} \in E$ the column associated with e is zero except for the u and v entries and $\{B_{\{ue\}}, B_{\{ve\}}\} = \{1, -1\}$). If $M_V \in R^{n \times n}$ and $M_E \in R^{m \times m}$ are positive definite matrices that define inner products on the vertices and edges respectively, the inner product Laplacian is defined as

$$M_V^{-1/2} B M_E B^T M_V^{-1/2},$$

where $M_V^{1/2}$ is the unique (symmetric) positive definite square root of M_V .

Open Questions.

As a consequence of the relationship between the inner product conductance and the spectrum of the inner product Laplacian, the standard approaches from spectral clustering can be lifted directly, and in some cases highly efficiently, to identify approximately optimal inner product conductance sets. Specifically, one can compute a k -dimensional embedding of the vertices based on the eigenvectors for the k smallest non-trivial eigenvalues of the inner product Laplacian and perform geometric clustering on the embedding vectors. Since the inner product Laplacian incorporates the structure and metadata into a single matrix, the resulting vertex clustering is a true fusion of the combinatorial and semantic data. Furthermore, in most cases this clustering cannot be pushed to only be representative of one of the combinatorial or metadata structure.

Unlike the normalized and combinatorial Laplacian, even with fixed inner products, there is potential that these embeddings are not uniquely defined up to symmetries of R^k (even when the eigenvalues are distinct). In particular, if M_E is not a diagonal matrix then the choice of signing on B is known to effect both the spectrum of the inner product Laplacian and its eigenvalues. Thus, in order to provide context for the derived clustering it is necessary to understand the effect the choice of signing of B has on the spectral embedding. Currently, nothing is known about the statistics of the perturbed spectrum and vertex embedding, either experimentally or theoretically.

Problem Statements.

I. Spectral Distribution: Suppose $G = (V, E)$ is a connected graph and M_V and M_E are the matrices for inner product spaces defined in V and E . Suppose that the signing of B is chosen uniformly at random and let λ_2 denote the random variable for the least non-zero eigenvalue of the inner product Laplacian. What is the form of the distribution of λ_2 ? How does this distribution depend on the properties of G , M_V , and M_E ?

II. Embedding Distribution: Suppose $G = (V, E)$ is a connected graph and M_V and M_E are the matrices for inner product spaces defined on V and E . Suppose that the signing of B is chosen uniformly at random and let $v_2, \dots, v_{k+1}$ be eigenvectors for the k smallest non-zero eigenvalues of the inner product Laplacian. Define $w_i \in R^k$ as the embedding vector associated with vertex i , that is, $(w_i)_j = (v_{j+1})_i$ for $1 \leq j \leq k$. What is the form of the distribution of w_i for a fixed vertex i ? How is the distribution of w_i correlated with the distribution of w_j ? Does this correlation depend on the structure of G , M_V , or M_E ?

References.

S. G. Aksoy, and S. J. Young, “Re-imagining Spectral Graph Theory,”
<https://arxiv.org/abs/2504.10624>, 2025.

NSASAG 26-12 Multi-layer Network Clustering

Background.

Let $G = \{G_1, G_2, \dots, G_\ell\}$ be a multi-layer network, where each layer is a graph $G(V, E_k)$ that has a common vertex set, but a layer-specific edge set. Some of the layers may hold symmetric relationships (undirected networks) and some of them relationships that are not symmetric (directed networks). Each layer may or may not be fully connected. If we define the aggregated network G_{agg} to be the union of all edges in all layers, we assume that G_{agg} is weakly connected.

This problem is concerned with hierarchical clustering approaches to multi-layer networks of this form with two or more layers. In this context, a hierarchical community is a nested collection of vertex subsets, where each level is a full partition of V . In the lowest level, all vertices are their own communities and at the highest level, all vertices are in a single community (for a weakly connected multi-layer graph). A noise or outlier vertex subset is possible at each level, representing vertices that are not plausibly linked to any other vertex until a higher level (representing weaker connections) is considered.

As a concrete example, consider clustering research articles within a citation + co-authorship network, whose vertices are research articles and layers represent different forms of relationships between the articles. The first layer is a co-authorship layer, where an edge between two articles means that the articles share one or more authors (this is a symmetric relationship). The second layer is a citation layer, where a directed edge from a source article to a target article means that the source cited the target (this is not typically a symmetric relationship, as the articles are typically published at different times). Several other types of relationships between research articles could be employed (such as sharing a funding source or a keyword) amounting to more network layers, but for most of this problem two layers are sufficient to consider. There are many such real-world and open research article datasets available (<https://github.com/allenai/s2and>).

There are several approaches in the literature addressing clustering on multi-layer networks (see the surveys Kim (2015) and Yousefzadeh (2025)). Firstly, one could just apply standard graph clustering to G_{agg} (with all layers treated equivalently, directed or not, possibly with weights and other variations). An opposite approach would be to apply a bespoke hierarchical clustering to each layer independently and then combine the output of each independent clustering to get a final clustering. The bespoke clustering could be very different for each layer (for example one layer could employ hypergraph clustering, whereas another layer could employ directed graph clustering). We observe that either approach will be inadequate when applied to datasets that have some community structures best resolved within one layer, a second class of structures best resolved in the other, and a third class best resolved within both layers jointly. We want an approach that is flexible to many common situations likely present in real-world datasets, as described below.

In the context of our example, assuming a cluster should contain sets of articles with pair-wise similarities that are statistically significant, consider the many possibilities of pairs of research articles (a, b) that would ideally be clustered together. We list some scenarios here:

1. Article a and article b both have many authors, and nearly all of them in common.
2. Article a and article b cite nearly identical sets of other articles.
3. Article a and article b are cited by nearly identical sets of other articles.
4. Article a and article b have a few authors in common (but not enough to declare they are similar) and have a few citations in common (but not enough to declare they are similar) yet the information jointly means they are similar enough to be in the same cluster.

Moreover, when there are similarities within latent group structures associated with any of 1-4, we also are interested in having such pairs be within the same cluster. For example, article a and article b have citations to nearly similar types of articles (as opposed to nearly identical citation sets).

We note that several additional challenges are possible in real-world datasets, including (i) articles can have low numbers of authors (single authors are common, but there might be anonymous articles, too), (ii) some authors might only author one paper in the dataset, for various reasons (it is outside of their area or they only published one thing to date), (iii) authors might use different names, (iv) an extremely large number of authors are listed (say 100's), (v) articles have very little citation structure (no one cites the article to date, or the article did not really cite any other paper), (vi) a very noisy citation structure exists (there is a large number of references in an article, or an extreme number other articles cite a popular article).

Lastly, it is typical that a small set of labeled vertices are a portion of some example communities (the labels are vertex pairs that ideally are deemed to be within the same community) and therefore we discuss *semi-supervised clustering* approaches. However, it is often the case that we wish to detect community structure that does not directly involve the labeled vertex pairs, where sought communities share similar connectivity properties to communities with labels.

Problem Statements.

- A. Design a semi-supervised hierarchical approach that, given a set of pair-wise labeled vertices that are desired to be within the same community, discovers *local* community structure at various scales. It should function well when data has various challenges described above, 1-4 and (i)-(vi). The output should be information that includes which vertices are contained in each cluster level. Your approach likely needs to adjust parameters based on graph densities present in the region of your input vertices, and

this information is useful output as well as it describes relative densities present within various network layers.

- B. Consider a multi-layer model that captures the various layers, group structure, and various complications described above, including 1-4, (i)-(vi), and combining both undirected and directed layers. For example, if utilizing stochastic block models (SBMs), then something akin to undirected and directed SBMs, degree-corrected SBMs, and noise modeling should be utilized. Use the model to analyze your approach in part A. Are there any guarantees your approach offers for parameter regimes in the model?
- C. Can the information output by your approach be used to reveal communities that are similar in nature but do not involve the pair-wise labeled vertices (elsewhere in the graph, or on a different graph with similar properties)?
- D. Consider a network with many more than 2 layers (e.g. articles sharing a funding source and/or weight similarity in article abstracts, via topic embedding, et cetera). Are there choices in your algorithmic approach in A. that make it computationally expensive to perform with many more than 2 layers. If so, are there adjustments or alternative approaches that are more tractable?

References.

J. Kim, J. G. Lee, “Community Detection in Multi-Layer Graphs: A Survey”, *SIGMOD*, 2015.

N. Yousefzadeh, “A Comprehensive Survey on Multi-Layer Graph Embedding Methods”, *Vietnam Journal of Computer Science (Open Access)*, 2025.

<https://github.com/allenai/s2and>.